One Spend - Release cheat sheet

tool/release.ps1 | version lives in pubspec.yaml as MAJOR.MINOR.PATCH[-alpha.N|-beta.N]+CODE | run from PowerShell in the repo root

The golden rule

Committing code never changes the version. Commit, push and merge PRs as often as you like. Run the release script **only** when you are about to upload to Play Console.

Version anatomy

Part	Meaning	Play track
MAJOR	Big redesign or breaking change	-
MINOR	New features	-
PATCH	Bug fixes only	-
`-alpha.N`	Early test build	Internal / Closed testing
`-beta.N`	Wider test build	Open testing
no suffix	Real release	Production
`+CODE`	Android versionCode. +1 on every script run (automatic)	-

Workflow A - everyday work (a fresh commit)

```
git checkout main; git pull
git checkout -b feat/my-change
# ...code...  (flutter run)
flutter analyze
flutter test
git add -A; git commit -m "feat: my change"
git push -u origin feat/my-change
# open PR -> merge.  No version bump.
```

Workflow B - shipping a build to Play

```
git checkout main; git pull
git status                      # must be clean
.\tool\release.ps1 -Bump patch  # pick from the table ->
# script pauses: replace 'TODO: what changed' in
#   lib/core/config/changelog.dart, save, press Enter
git push --follow-tags
```

Then upload **`build/app/outputs/bundle/release/app-release.aab`** to the Play Console track the script prints.

Which command? (every scenario)

Scenario	Command	Example
First ever upload of the current version	**`-Rebuild`**	`alpha.1+1 -> alpha.1+2`
Another alpha, same version	**`-Pre alpha`**	`alpha.1 -> alpha.2`
Move alpha to beta	**`-Pre beta`**	`alpha.2 -> beta.1`
Fix a bug found during beta	**`-Pre beta`**	`beta.1 -> beta.2`
Beta is good, ship to Production	**`-Promote`**	`1.0.0-beta.2 -> 1.0.0`
Bug fix after a Production release	**`-Bump patch`**	`1.0.0 -> 1.0.1`
Bug fix you want tested first	**`-Bump patch -Pre beta`**	`1.0.0 -> 1.0.1-beta.1`
New features (tested first)	**`-Bump minor -Pre beta`**	`1.0.1 -> 1.1.0-beta.1`
New features, straight to Production	**`-Bump minor`**	`1.0.1 -> 1.1.0`
Big redesign	**`-Bump major -Pre beta`**	`1.x -> 2.0.0-beta.1`
Play rejected it / re-upload same version	**`-Rebuild`**	`same name, CODE+1`
APK instead of .aab	**`add -Target apk`**	
Debug APK for your phone (changes nothing)	**`-Debug`**	
Try the script without building	**`add -SkipBuild`**	
Notes already written, don't pause	**`add -Yes`**	
Show help	**`-Help`**	

Prefix every command with **`.\tool\release.ps1`**. Cannot go from beta back to alpha. `-Promote` only works on an alpha/beta; `-Pre` alone only works on an alpha/beta (use -Bump after a real release).

Typical life of a version

```
-Bump minor -Pre beta   1.0.0        -> 1.1.0-beta.1
-Pre beta               1.1.0-beta.1 -> 1.1.0-beta.2   (fixes)
-Promote                1.1.0-beta.2 -> 1.1.0          (ship)
-Bump patch             1.1.0        -> 1.1.1          (hotfix)
```

One Spend - Release cheat sheet

tool/release.ps1 | version lives in pubspec.yaml as MAJOR.MINOR.PATCH[-alpha.N|-beta.N]+CODE | run from PowerShell in the repo root

What the script does, in order

1. Checks the tree is clean 2. Bumps pubspec.yaml 3. Adds a What's new entry and pauses for your notes 4. Runs the version test 5. Commits `chore(release): X (CODE)` 6. Builds the signed .aab 7. Tags vX

When things go wrong

"Commit or stash your changes first"
The script only runs on a clean tree.

```
git status
git stash        # or commit your work
```

Script blocked by execution policy
Run it through PowerShell with a bypass:

```
powershell -ExecutionPolicy Bypass -File tool/release.ps1 -Bump patch
```

Build failed after the bump
The release commit was undone; pubspec + changelog are still staged. Keep your notes and retry:

```
git restore --staged --worktree pubspec.yaml
git commit -m "docs: release notes" lib/core/config/changelog.dart
# fix the build, then run the same release command again
```

Undo a release you haven't pushed
e.g. after a -SkipBuild trial. reset --hard drops uncommitted work, so check git status first.

```
git tag -d v1.0.1
git reset --hard HEAD~1
```

"Tag vX already exists"
That version was already released. Use the next bump, or -Rebuild for a re-upload.

"Release notes are still TODO"
Nothing was changed. Write the notes when the script pauses (or drop -Yes).

"android/key.properties is missing"
Build is signed with the DEBUG key and Play will reject it. Set up the upload key first.

app_version_test fails
pubspec version and newest changelog entry differ. Don't hand-edit the version; let the script do it.

Rules to remember

- **Never edit `version:` by hand.** The version test breaks if pubspec and What's new disagree.
- **Don't pre-add next version's changelog entry.** Same test. Let the script add it and pause.
- **Every upload needs a new CODE.** Re-uploading = `-Rebuild`, never an old .aab.
- **Release from main** after pulling the merged PRs. On another branch the script just warns.
- **Always push the tag**: `git push --follow-tags`.

> **If main becomes protected (PRs required):** `git push --follow-tags` will be rejected. Run the release on a branch, merge it with a normal merge (not squash, so the tag stays on main's history), then `git push origin v1.0.1`.

Day-to-day commands

Task	Command
Install packages	`flutter pub get`
Run the app	`flutter run`
Lint	`flutter analyze`
All tests	`flutter test`
One test file	`flutter test test/features/x_test.dart`
One test by name	`flutter test --plain-name "..."`
Regenerate Drift code	`dart run build_runner build --delete-conflicting-outputs`
Current version	`Select-String '^version' pubspec.yaml`
List release tags	`git tag --sort=-v:refname`

Testing a real in-app update

Only works on a Play-installed build. Upload a build with a higher versionCode to Internal app sharing or an internal-test track, install the older one from Play, then Settings > About > Check for updates.